Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic_netops_validator_v5 for deterministic end-to-end correctness checks; preregistration used shared_initial_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model_calls_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.**

I. Introduction

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study's preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1).

A decisive preregistered design choice was shared_initial_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

II. Related Work

Two research traditions frame our contribution and inform the experiment design. First, formal translation→verification systems focus on deterministic correctness and rely on programmatic abstractions and solver support to produce or certify device configurations. Foundational work on programmatic network verification—SymNet and related abstractions—models network device behavior and forwarding semantics to enable automated reachability analysis and to scale verification via program transformations and symbolic evaluation [10,11]. Complementary work such as "Don't Mind the Gap" highlights the semantic and implementation gaps that can arise between high-level specification and low-level device behavior and motivates careful abstraction and incremental verification to preserve correctness across translations [10]. More recent formal languages and algebraic frameworks (e.g., Weighted NetKAT and related quantitative verification

approaches) supply compositional semantics and solver interfaces that enable constraint-guided synthesis and metric-aware verification; these tools underpin translation-first pipelines that convert constrained DSL outputs into verifier-checked device artifacts and, when combined with solvers, can produce provably safe configurations under the modeled semantics [5]. Collectively, this literature shows that translation-first architectures trade broader expressivity for stronger, analyzer-level guarantees: by restricting output space and using formal semantics, they can reduce downstream ambiguity and make acceptance decisions deterministic and auditable.

Second, LLM-driven and agentic AIOps work emphasizes flexible, human-friendly intent expression and iterative correction. Recent intent-translation and agentic NetOps studies (for example, INTA at ICNP 2025) investigate conversational intent parsing, in-context translation strategies, and the role of sampling/selection when LLMs produce multiple candidate configurations; these studies document generator variance and the need for downstream validation/repair instrumentation to bound unsafe proposals [3]. Parallel work on verifier-assisted agentic pipelines (e.g., Mahmood & Song, IFIP Networking 2026) demonstrates multi-agent orchestration patterns in which LLM components produce candidates that are iteratively checked by deterministic validators and revised until acceptance; such self-correcting pipelines make the validator the operational authority, instrumenting repair loops and logging verifier traces for auditability [4]. Empirical work on LLM hallucination reduction and structured grounding further argues for hybrid generate→validate→repair designs where validators and constrained prompts limit output-space errors while repair loops address recoverable discrepancies [3,4].

Comparative axes and residual gaps. Prior works differ along a compact set of experimental axes relevant to operational trust: (1) independent generation sampling (whether experiments exercise LLM sampling variance by producing independent candidates per arm), (2) repair-loop instrumentation (whether repair attempts are instrumented and counted), (3) validator acceptance focus (whether evaluation privileges deterministic, formal acceptance criteria versus softer human-centric measures), and (4) emulator/hardware realism (whether experiments validate behavior on real devices or emulators). Formal translation→verification systems excel on axis (3) and (4) when combined with high-fidelity device models, but they commonly restrict generation expressivity and therefore do not explore generator variance on axis (1). Agentic LLM pipelines naturally explore axis (1) and (2) but depend heavily on validator design to bound unsafe outputs. Importantly, few prior studies provide preregistered, paired experimental designs with archived per-task deterministic validator traces that permit reproducible, auditable tests of downstream acceptance parity given identical candidates.

Positioning of the present run. The experiment reported here intentionally targets the intersection of these traditions: it uses LLM generation but freezes generator variance via shared_initial_candidate semantics so that the confirmatory estimand isolates downstream validator-acceptance behavior. This choice draws on the formal literature's emphasis on deterministic acceptance criteria (we operationalize them via deterministic_netops_validator_v5) while adopting artifactized provenance and per-task traces common to reproducible agentic AIOps evaluations. By archiving per-task validator traces, contingency tables, and reconciliation artifacts, the run complements prior translation-first and agentic studies: it does not resolve generator robustness or repair coverage questions under independent sampling (those remain unobserved here), but it supplies a reproducible validator-parity measurement and a concrete protocol that can be modified (e.g., remove shared_candidate, enable repair instrumentation, increase model_calls_used) to exercise the axes left untested by prior work.

III. Methodology

This section expands the preregistered and executed methodology with concrete, artifact-grounded implementation and validation details. All referenced artifacts and parameter values are drawn from the archived execution and analysis bundle (paths cited where informative).

Preregistration and frozen contract. The study design was frozen in preregistration CAND-2026-001-prereg-v1. Key preregistered parameters used without post-hoc change: planned_episode_count = 240 (120 paired tasks), model scope = {"gpt-5-mini"}, execution adapter = hosted_netops_gvr_v1, generation semantics = shared_initial_candidate, maximum_model_calls = 240, and validator = deterministic_netops_validator_v5 (validator_version = 5.0). The preregistered analysis executor was paired_binary_analysis_v1 with bootstrap_resamples = 10000 and bootstrap_seed = 1729. The preregistration SHA recorded in the execution manifest is provided for audit.

Task generation and difficulty metadata. Confirmatory tasks were produced deterministically by deterministic_netops_task_generator_v5. Each manifest entry (execution/task_manifest.jsonl) contains (a) the human-readable intent, (b) canonical reference answer (expected sequence of configuration commands), (c) initial and expected final device states, (d) a required_sequence listing required field-level operations, and (e) difficulty metadata fields used for stratification: changed_interface_count, dependency_rule_count, level $\in$ {medium, hard, very_hard}, required_command_count, and transient_rule_count. Representative manifest entries (e.g., task-000001) are archived; these entries are the ground truth used by the deterministic validator's normalization and required-command checks.

Generation orchestration and shared_initial_candidate implementation. The orchestration implemented shared_initial_candidate as follows: for each paired task the adapter invoked a single LLM generation stage (stage key "shared_initial_generation") and persisted the textual candidate. The single stored candidate was then supplied verbatim (no per-arm regeneration) to both the translation-first and agentic downstream arms for

syntactic/semantic processing and deterministic validation. Execution accounting files record model_calls_used = 120 (one LLM call per task pair) and provider audit metadata (analysis/deterministic_reconciliation.json) shows hosted_model_call_event_count = 120, cache_miss_count = 120, and cross_condition_cache_key_reuse_observed = false, indicating each pair used an isolated model invocation without cross-condition reuse.

Model configuration and call semantics. The declared generation model is gpt-5-mini as recorded in execution/model_configuration.json. Execution parameters recorded in that file include max_output_tokens = 1500 and maximum_attempts_per_call = 1. Provider field 'openai_responses' documents the execution provenance; no retrieval augmentation was used (retrieval_augmented_generation = false in the preregistration). The single-call, single-candidate policy is consistent with the preregistered budget (model_calls_used <= maximum_model_calls) and with the planned shared_candidate estimand.

Deterministic validator: normalization, constraints, and scoring rules. The deterministic_netops_validator_v5 implements the preregistered full_intent_constraint_validation scorer. Validation is applied in two phases recorded in scoring JSONs: validation_before (candidate normalization and difficulty/context diagnostics) and validation_after (post-orchestration final state simulation and constraint checks). Concrete checks include: - Syntax and canonical normalization: compute normalized_response and canonical ordering when semantically reorderable commands exist; normalized_response is archived in execution/scoring/*.json. - Required command inclusion: verify that every entry in the task payload's required_sequence is present in the candidate (field-level coverage) yielding required_command_count and parsed_command_count. - Ordering and dependency checks: enforce patterns such as make-before-break (uplink switchover), shutdown_modify_restore (MTU/VLAN changes requiring admin down), and group constraints (minimum_active_in_groups, final_group_constraints). Each pattern maps to validator rules that set violation_codes and violation_count. - Final state simulation: apply the candidate commands to a deterministic emulator model of device states and check that the resultant final_state matches expected_final_state for the fields constrained by the task, producing validation_after.valid boolean recorded per episode.

Scoring semantics and success definition. Per preregistration, a task episode is scored success (binary 1) iff validation_after.valid == true and scoring_rule == "full_intent_constraint_validation". The scoring JSONs include score_reason_code (e.g., VALID_CONFIGURATION) and repair_applied flag. All per-episode scoring artifacts (execution/scoring/task-000***-*.json) are the canonical record of the validator decision and include raw_response_sha256, shared_initial_candidate_sha256, and validator_trace with parsed counts and normalized configurations.

Repair policy and instrumentation. The preregistered transformation_scope allowed a single repair iteration per task (maximum_repair_calls_per_task = 1). The orchestration records repair_provider_trace_path and repair_provider_trace_sha256 when repair is requested and applied. In this execution, aggregated repair_applied count = 0 across execution/scoring/*.json (repair_applied == false for every episode), therefore the repair loop remained uninvoked; this observation is archived and referenced in analysis artifacts.

Contamination checks, missingness, and treatment rules. The preregistered contamination_plan included (i) a simulator fidelity suite that compares emulator outputs to held-out public device examples and flags tasks with >5% disagreement, (ii) grammar coverage/unit tests for vendor CLI constructs, and (iii) execution isolation (explicit model context resets). Analysis/contamination_summary.csv is empty for this run (no flags). Missingness rules allowed two retries on transient provider failures and specified exclusion only when both pipeline runs were lost for a pair; the execution manifest reports failed_episode_count = 0 and missingness_summary indicates complete_pairs = 120.

Analysis executor and inferential procedures. Analysis followed paired_binary_analysis_v1 exactly as preregistered. The executor computed the 2×2 paired contingency table (n11,n10,n01,n00), the paired difference in success rates (A - B), bootstrap percentile 95% CI using bootstrap_resamples = 10000 and bootstrap_seed = 1729 (both recorded in analysis/analysis_log.jsonl), and an exact McNemar two-sided test for the paired binary outcome. Analysis artifacts (analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl) and deterministic_reconciliation.json provide reproducible inputs and outputs for independent recomputation.

Reproducibility and artifact references. Every described artifact is archived in the execution bundle and referenced in the execution manifest and analysis artifacts. Important file paths used by reviewers/readers: execution/execution_manifest.json (execution accounting and preregistration_sha256), execution/model_configuration.json (gpt-5-mini declaration), execution/task_manifest.jsonl (task payloads and difficulty fields), execution/responses/* (shared and per-condition textual candidates), execution/scoring/*.json (per-episode validator traces and scoring decisions), analysis/paired_contingency_table.csv and analysis/condition_summary.csv (primary outcomes), and analysis/analysis_log.jsonl (bootstrap parameters). These artifacts collectively document the operational semantics: single generation per pair $\rightarrow$ identical candidate delivered to both arms $\rightarrow$ deterministic normalization and constraint validation $\rightarrow$ binary success decision archived per episode.

Protocol limitations relevant to methodology. The shared_initial_candidate semantics were a preregistered design choice and implemented deterministically; it removes generator sampling variance and the operational effect of independent repairs. The methodology above therefore describes an exactly specified validator-acceptance parity

measurement: it assesses downstream orchestration and validator agreement on identical textual candidates rather than independent generator robustness. Practical follow-ups to test independent generation and repair require concrete protocol changes (remove shared_candidate, increase model_calls_used to generate independent candidates per arm, enable repair calls) that are specified in the manuscript's Discussion and documented in the preregistration as amendable, auditable modifications.

IV. Results

Execution accounting and primary numeric outcomes. The execution manifest (execution/execution_manifest.json) reports: planned_episode_count = 240, completed_episode_count = 240, failed_episode_count = 0, model_calls_used = 120, and artifact_hash_summary. Deterministic reconciliation (analysis/deterministic_reconciliation.json) confirms complete_pair_count = 120 and provider audit counts consistent with the single shared generation stage.

Deterministic scoring and contingency. The primary contingency (analysis/paired_contingency_table.csv) is: • n11 = 120 (accepted by both arms) • n10 = 0 (accepted only by guarded) • n01 = 0 (accepted only by baseline) • n00 = 0 (rejected by both) Marginal success rates are baseline = 120/120 = 1.0 and guarded = 120/120 = 1.0. Paired difference (A - B) = 0.00. The bootstrap percentile 95% CI (10,000 resamples, seed 1729) reported in analysis/analysis_log.jsonl is [0.00, 0.00], and the exact McNemar two-sided p = 1.0.

Repair and secondary outcomes. Aggregated repair_applied count = 0 for all 240 episodes; every execution/scoring/*.json entry shows repair_applied = false. Therefore preregistered secondary estimands about repair coverage (H2) are unobserved in this run. Representative per-task traces (e.g., execution/scoring/task-000001-baseline.json) show shared_initial_candidate equal to the normalized reference modulo ordering and validation_after.valid = true.

Contamination and missingness diagnostics. The preregistered contamination checks produced no flags: analysis/contamination_summary.csv is empty and analysis/missingness_summary.csv records complete_pairs = 120 with no observed failures. Provider metadata reports cache_miss_count = 120 and cross_condition_cache_key_reuse_observed = false, supporting execution isolation and adherence to the contamination_plan.

Representative example. Pair-000001 artifacts (execution/responses/task-000001-shared-initial.txt and execution/scoring/task-000001-baseline.json) document the shared candidate, normalized_response, parsed_command_count = 4, required_command_count = 4, and validation_after.valid = true. Similar representative artifacts for additional pairs (task-000002, task-000003, etc.) are archived and referenced in the execution manifest.

V. Discussion

What the run measures — and what it does not. Because shared_initial_candidate semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance (n11 = 120) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Interpretation of perfect concordance. The artifact bundle permits an evidence-grounded interpretation of the degenerate concordance. Per-task scoring artifacts (execution/scoring/task-000XXX-*.json) repeatedly show: (i) the shared_initial_candidate equals the normalized_response passed to the validator; (ii) parsed_command_count equals required_command_count for successful items; and (iii) validation_after.valid == true with violation_count == 0. These per-task diagnostics explain the observed n11 = 120: for every paired task the single generated candidate satisfied the deterministic_netops_validator_v5 acceptance criteria (syntactic normalization, inclusion of required commands, ordering/dependency rules, and final state consistency). Those observations are empirical facts recorded in the archived scoring JSONs (representative examples: task-000001, task-000002, task-000003). The results therefore document downstream agreement on identical inputs, not resilience to generator errors or repair dynamics.

Artifact-grounded diagnostics. The execution and analysis manifests provide multiple, cross-checked traces that constrain plausible explanations without speculation. analysis/deterministic_reconciliation.json documents that provider calls were isolated (hosted_model_call_event_count = 120, cross_condition_cache_key_reuse_observed = false). analysis/contamination_summary.csv and analysis/missingness_summary.csv show no contamination or missingness flags. analysis/analysis_log.jsonl records bootstrap parameters used to compute the degenerate CI [0.00, 0.00] (bootstrap_resamples = 10000, seed = 1729). The absence of repair_applied flags across execution/scoring/*.json (repair_applied == false) is also a recorded fact that explains why preregistered secondary estimands about repair coverage are uninformative here.

Design lessons for discriminating comparisons. The run's restricted estimand was intentional and useful for validator-regression or orchestrator-equivalence checks. To discriminate independently between architectures along the originally envisaged axes (generator correctness, repair coverage, robustness to hallucination), the preregistered protocol modifications are necessary and mechanically verifiable in the artifact bundle:

- Independent generation per arm: remove shared_initial_candidate so each arm receives its own LLM generation. This change predictably increases model_calls_used from 120 to 240 for a single initial candidate per arm and will appear in execution/model_configuration.json and execution/execution_manifest.json as higher hosted_model_call_event_count.

- Sampling multiple generations per arm: draw k > 1 independent initial samples per arm to estimate generator variance; recorded artifacts will include multiple responses per task (execution/responses/*) and nontrivial dispersion in parsed_command_count and validation outcomes.

- Enable and instrument repair loops: permit repair iterations and record repair_provider_trace_path and repair_prompt_sha256 in execution/scoring/*.json; aggregating repair_applied counts and mean repair iterations becomes possible and directly measurable by the analysis executor.

- Larger / hardware-in-the-loop fidelity: add held-out simulator↔hardware fidelity checks from the contamination_plan to reduce simulator bias; disagreement rates would appear in analysis/contamination_summary.csv and could trigger preregistered exclusions.

All of these modifications are concrete, preregistrable, and observable in the archived manifests, which makes future discriminating experiments auditable.

Threats to validity — elaborated. Internal validity: the shared_candidate design intentionally removes generator variance and thus trades the ability to test generator or repair effects for reduced sampling noise in downstream validator comparison. Execution manifests confirm no post-hoc reruns or cross-condition cache reuse. Construct validity: deterministic_netops_validator_v5 operationalizes a conservative and automated correctness notion (full_intent_constraint_validation). That binary acceptance criterion is narrow by design and omits qualitative operator concerns (explainability, maintainability). External validity: the task corpus was synthetic (deterministic_netops_task_generator_v5) and executed in Mininet-derived emulation; emulator-to-hardware divergence remains a documented residual uncertainty in the contamination_plan. Conclusion validity: degenerate contingency (all concordant) compresses variance estimates to a point mass; the CI and McNemar p correctly summarize the observed data but do not imply equivalence under independent operation.

Operational implications. For practitioners wanting to validate or regress deterministic validators and orchestrators, the shared_candidate protocol is a low-variance, resource-efficient option: it isolates downstream differences while keeping model call costs low. For organizations comparing architectures for independent correctness or repair behavior, the artifact bundle shows how to extend the protocol (independent sampling, larger model_call budgets, repair instrumentation) and what artifacts will evidence claims about generator robustness or repair coverage. The run's recorded metadata (execution/model_configuration.json declaring gpt-5-mini, analysis/deterministic_reconciliation.json provider audit counts) provides a minimal reproducible template for practitioners to replicate the validator-parity check before investing in costlier independent-generation experiments.

Reproducibility checklist for follow-ups. Based on archived manifests, a reproducible discriminating experiment must (at minimum) include: (1) explicit preregistration of generation semantics (shared vs independent) and expected model_call budget; (2) per-task recording of all model responses (execution/responses/*) and repair traces (repair_provider_trace_path) when repair is enabled; (3) contamination and fidelity checks per contamination_plan with documented thresholds for exclusion; (4) analysis executor parameters (bootstrap_resamples, seed) recorded in analysis/analysis_log.jsonl; and (5) versioned identifiers for task generator and validator to ensure deterministic task/validator behavior across runs. All these items are present in the current artifact bundle and are sufficient to audit whether a subsequent run implemented the intended discriminating design.

Summary. The run produced a clear, artifact-grounded answer to the narrower question it preregistered: validator-acceptance parity for identical LLM candidates across two orchestration pipelines. The archived execution and analysis artifacts support transparent interpretation of that answer, document why secondary estimands (repair coverage) were unobserved, and provide a concrete, verifiable pathway to modify the protocol for discriminating tests of generator and repair behavior.

VI. Limitations

- Preregistered shared_initial_candidate semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic_netops_validator_v5) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (repair_applied = false in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

VII. Conclusion

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared_initial_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair (n11 = 120, n10 = n01 = n00 = 0),

yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar p = 1.0). No repairs were applied (repair_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

Disclosure Statement

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared_model_version in execution/model_configuration.json); provider record 'openai_responses'. The hosted adapter family was hosted_netops_gvr_v1. Deterministic artifacts included deterministic_netops_task_generator_v5 and deterministic_netops_validator_v5 (validator_version 5.0). The run deliberately used the preregistered shared_initial_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model_calls_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired_binary_analysis_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

References

[1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909

[2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881

[3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391

[4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993

[5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318